

URL Risk Checker

A Manifest V3 Browser Extension for Phishing & Scam-Site Warnings

Project Documentation

Part 1: Guard.io Product Research | Part 2: Build Extension

Table of Contents

Executive Summary

Part 1 — Product Research: Guard.io

- 1.1 Research Approach
- 1.2 What Guard.io Does
- 1.3 Why It Has Been Successful
- 1.4 Tech Stack Assessment (Inferred)
- 1.5 Team Structure
- 1.6 Feasibility Assessment
- 1.7 Proposed Build Plan for a Guard.io-Comparable Product
- 1.8 How AI Tools Accelerate This

Part 2 — Build: URL Risk Checker Extension

- 2.1 Objective & Scope
- 2.2 AI Tools Used Across the Project
- 2.3 Technology Stack
- 2.4 Dependencies
- 2.5 Development Environment
- 2.6 System Architecture
- 2.7 Backend Implementation
- 2.8 Threat-Intelligence Sources
- 2.9 Risk Scoring Logic
- 2.10 Chrome Extension Implementation
- 2.11 Manifest V3 Design Considerations
- 2.12 Setup & Run Instructions
- 2.13 AI-Assisted Development Notes
- 2.14 Known Limitations
- 2.15 What Would Change for Production
- 2.16 30-Day MVP Roadmap
- 2.17 Time Spent & Attribution

Appendix A — Source Repository

Executive Summary

This document covers two parts of an assignment: (1) product research into Guard.io, a consumer cybersecurity company whose browser extension and mobile app protect users from phishing and scam sites, and (2) the design and build of a small proof-of-concept Chrome extension — the URL Risk Checker — that demonstrates the core mechanic behind that kind of product: reading the page a user is on, checking it against real threat-intelligence sources, scoring the risk, and warning the user before harm occurs.

The build uses a Manifest V3 Chrome extension paired with a Node.js/Express backend. The backend queries two independent, legitimate threat-intelligence providers — Google Safe Browsing and VirusTotal — combines their findings with a small set of local heuristics, and returns a risk score and status (safe / suspicious / malicious). The extension intercepts navigation in the background and redirects the tab to a warning page before the risky page is shown, rather than only checking on demand.

AI tools were used throughout: Perplexity for the initial Guard.io product research, ChatGPT for early architecture exploration (including the decision to move the backend from ASP.NET Core to Node.js/Express), and Claude for the majority of the hands-on implementation, debugging, and this documentation. Section 3.2 and 3.13 describe this in detail, including what was reviewed, changed, or rejected.

This is a proof-of-concept, not a production system. Section 3.14 and 3.15 list the specific gaps (no caching layer, no request timeouts, open CORS policy, no automated false-positive/negative testing) and what would need to change before this could run in production.

Part 1 — Product Research: Guard.io

1.1 Research Approach

Guard.io does not publish its internal architecture, team structure, or proprietary detection logic, and this research deliberately relied only on publicly available information — the product's marketing site, its public-facing behavior, and general industry knowledge about how consumer anti-phishing products are typically built. No attempt was made to access source code, bypass systems, or obtain confidential information, in line with the assignment's constraints.

Perplexity (an AI search tool that cites live web sources) was used as the primary research tool for this section, since the task benefits from a tool that can pull and synthesize current public information rather than rely on a fixed knowledge cutoff. Findings below are a synthesis of that research together with general, publicly-known patterns in the browser-security-extension space.

1.2 What Guard.io Does

Guard.io is a consumer-facing cybersecurity product, delivered primarily as a browser extension (with a companion mobile app), that protects everyday users — not security professionals — from phishing sites, scam pages, malicious downloads, and other online fraud in real time as they browse. Its core promise is simple: warn people before they hand over a password or payment details to a fake site, without requiring any technical understanding from the user.

- **Real-time link and page protection:** checks pages as they load and blocks or warns on known-bad destinations.
- **Identity and data-breach monitoring:** alerts users if their email, phone number, or accounts appear in known breaches.
- **Simple, non-technical UX:** a colored badge and plain-language warning rather than a technical threat report.
- **Broad device coverage:** browser extension plus mobile app, so protection follows the user across devices.

1.3 Why It Has Been Successful

Based on the research, Guard.io's success is less about having a uniquely superior detection algorithm and more about product and distribution execution in a category most competitors treat as an enterprise/IT problem rather than a consumer one:

- It targets non-technical consumers directly, rather than assuming an IT department is protecting the user, which is the assumption most enterprise security tools make.
- The warning experience is deliberately simple — a clear stop/warning screen rather than a technical alert full of jargon — which builds trust with a non-expert audience.
- It combines multiple protection surfaces (browsing, identity/breach monitoring, mobile) into one subscription, increasing perceived value and reducing churn.
- Low-friction installation (a browser extension, not a heavyweight security suite) lowers the barrier to adoption for consumers.

1.4 Tech Stack Assessment (Inferred)

Guard.io's exact internal stack is not publicly disclosed, so this is an informed inference based on what is observable from the outside and typical patterns for this category of product, not confirmed fact:

Layer	Likely Approach (inferred)
Browser extension	Manifest V3 (Chrome, Edge), likely also Firefox/Safari variants, built in JavaScript/TypeScript
Detection signal sources	A blend of licensed/partnered threat-intelligence feeds plus a proprietary database built from their own user base and crawling
Backend	A cloud-hosted API service (provider unknown) fronting the detection logic, likely with heavy caching given real-time latency requirements
Detection logic	Rules and reputation lookups for known threats, plus heuristic/ML models for new/unseen phishing pages (domain age, page structure, brand-impersonation signals)
Mobile app	Native iOS/Android apps for link scanning, breach alerts, and account management

This should be read as an educated estimate, not a verified fact — Guard.io has not published architecture details, and none were obtained here.

Part 2 — Build: URL Risk Checker Extension

2.1 Objective & Scope

Build a small Manifest V3 Chromium extension that reads the current URL, checks it against at least two legitimate threat-intelligence sources, produces a simple risk score, and warns the user when the URL is flagged as risky — while staying invisible during normal, safe browsing.

This is a proof-of-concept demonstrating the core mechanic of a product like Guard.io, built end-to-end: extension, backend, two real threat-intelligence integrations, and a warning flow. It is explicitly not a production-hardened security product — see Section 2.14 for known limitations.

2.2 AI Tools Used Across the Project

Tool	Role in this project
Perplexity	Part 1 product research on Guard.io (Section 1).
ChatGPT	Early architecture exploration for the extension/backend split, and the decision to pivot the backend from an initial ASP.NET Core (Clean Architecture + MediatR) design to a Node.js + Express implementation.
Claude	The majority of hands-on implementation: backend file scaffolding, the Google Safe Browsing and VirusTotal integrations, the risk-scoring logic, the Chrome extension (manifest, popup, background service worker, warning page), debugging, and this documentation.

Section 2.13 below goes into more detail on how AI assistance was structured, reviewed, and where AI-suggested approaches were changed or rejected during development.

2.3 Technology Stack

Layer	Technology
Browser extension	JavaScript, Chrome Extension Manifest V3
Backend runtime	Node.js
API framework	Express.js
Outbound HTTP calls	Axios
Environment configuration	dotenv
Cross-origin support	cors
Dev tooling	nodemon (auto-restart during development)
Package manager	npm
Code editor	Visual Studio Code

2.4 Dependencies

Package	Type	Purpose
express	Runtime dependency	Web framework — routing, middleware, HTTP server
axios	Runtime dependency	Outbound HTTP requests to Google Safe Browsing and VirusTotal
dotenv	Runtime dependency	Loads API keys and config from a local .env file
cors	Runtime dependency	Allows the extension's cross-origin requests to reach the API
nodemon	Dev dependency	Auto-restarts the server on file changes during development

2.5 Development Environment

- Code editor: Visual Studio Code
- Runtime: Node.js (npm-managed project, package.json + package-lock.json)
- Browser: Google Chrome, loaded as an unpacked extension via chrome://extensions (Developer Mode)
- Local backend: runs on http://localhost:3000 during development
- Secrets: stored in a local .env file (excluded from version control via .gitignore)

2.6 System Architecture

The extension is a thin client. All threat-intelligence lookups, scoring logic, and API keys live server-side in the Node.js backend — the extension only ever talks to that backend, never directly to Google or VirusTotal. This keeps API keys out of the browser entirely and keeps the scoring logic in one testable place.

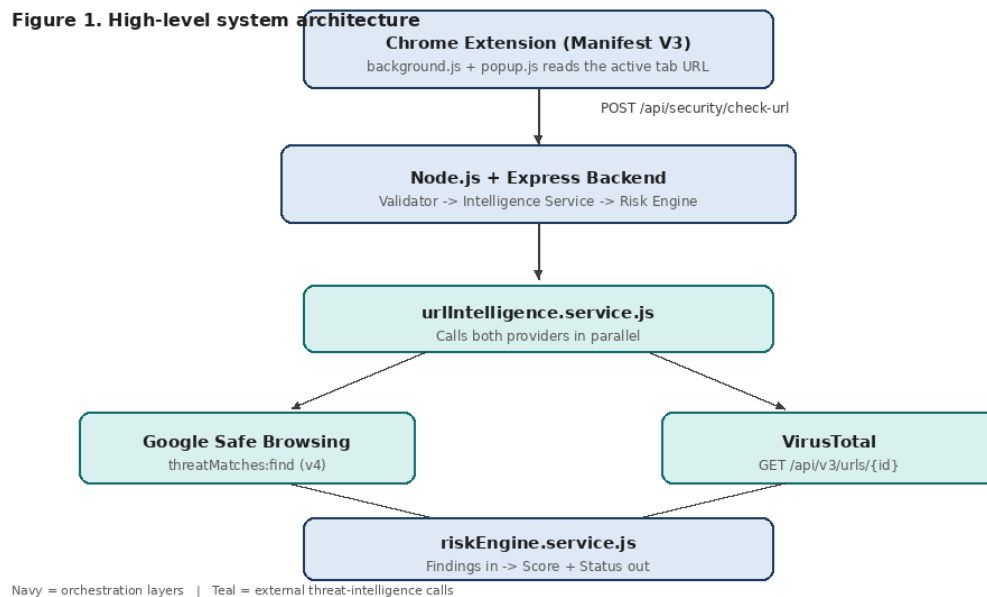


Figure 1. High-level system architecture.

Request flow, at a glance:

- The background service worker detects a navigation and reads the destination URL.

- It sends that URL to the backend's POST `/api/security/check-url` endpoint.
- The backend validates the URL, then queries both threat-intelligence providers in parallel.
- A risk engine combines their findings (plus a few local heuristics) into a score and a status: safe, suspicious, or malicious.
- If risky, the extension redirects the tab to a bundled warning page before the real page is shown. If safe, browsing continues with no visible interruption.

2.7 Backend Implementation

The backend follows a simple layered structure — routing, controller, validator, provider adapters, an aggregation service, and a pure risk-scoring engine — so that each responsibility is isolated and independently testable.

Figure 2. Backend request flow, file by file

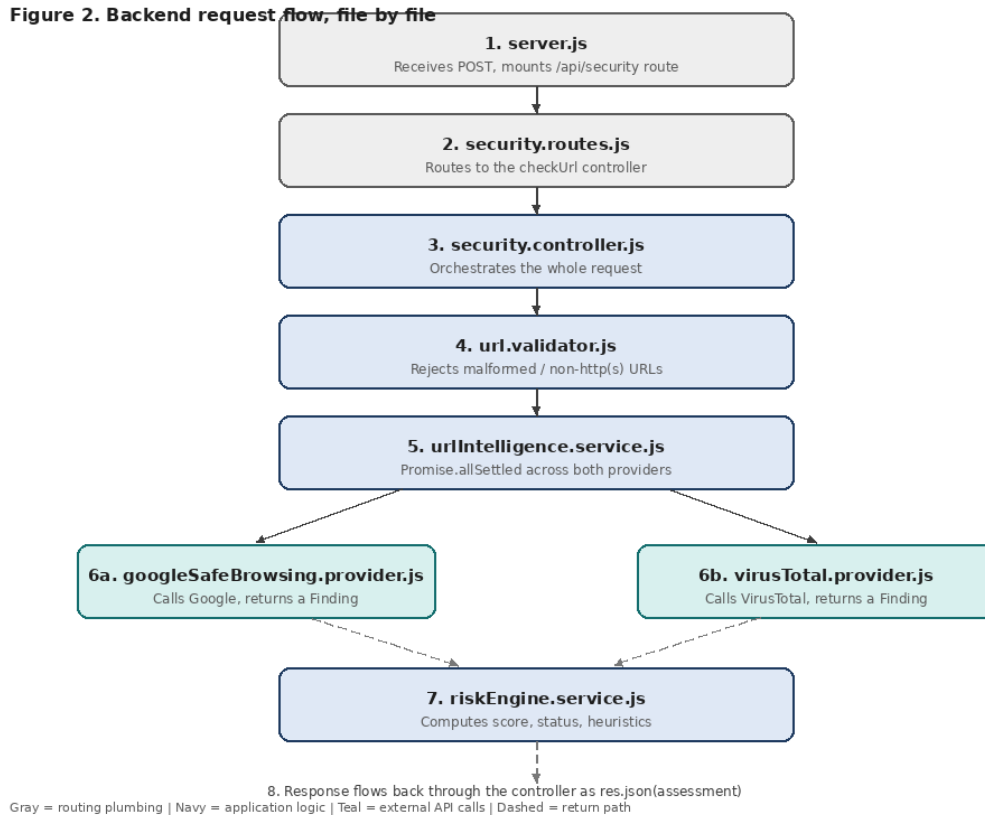


Figure 2. Backend request flow, file by file.

File	Responsibility
src/server.js	Express app entry point; wires middleware and mounts the /api/security route.
src/routes/security.routes.js	Defines POST /check-url and points it at the controller.
src/controllers/security.controller.js	Orchestrates one request: validate -> fetch findings -> score -> respond. Contains no business logic itself.
src/validators/url.validator.js	Rejects missing, non-string, or non-http(s) input before any external API call is made.
src/services/urlIntelligence.service.js	Calls both provider adapters in parallel via Promise.allSettled and normalizes their results.
src/providers/googleSafeBrowsing.provider.js	Adapter for the Google Safe Browsing v4 API.
src/providers/virusTotal.provider.js	Adapter for the VirusTotal v3 API.
src/services/riskEngine.service.js	Pure function: takes findings in, returns a score, status, and heuristic notes. No network calls.
src/config/env.js	Loads and exposes API keys and port config from .env.

A deliberate design choice worth calling out: the risk-scoring logic (riskEngine.service.js) never imports or calls the provider adapters, and the provider adapters have no knowledge that scoring exists. The controller is the only piece that knows about both. This keeps the scoring rules testable with a hand-built array of findings and no network mocking, and means either side can change independently.

2.8 Threat-Intelligence Sources

Two independent, publicly available threat-intelligence sources were integrated, satisfying the assignment's requirement of at least two sources. Using Promise.allSettled means a failure or slowdown in one source degrades gracefully rather than failing the whole request.

2.8.1 Google Safe Browsing (v4)

- Endpoint: POST <https://safebrowsing.googleapis.com/v4/threatMatches:find>
- Authentication: API key passed as a query parameter, restricted in Google Cloud Console to the Safe Browsing API only.
- Request: the current URL plus the threat types checked (malware, social engineering / phishing, unwanted software, potentially harmful applications).
- Response: an empty body means the URL is not on any known threat list; a matches array means it is a known threat, and the array lists why.

2.8.2 VirusTotal (v3)

- Endpoint: GET <https://www.virustotal.com/api/v3/urls/{id}>, where {id} is the URL, base64url-encoded.
- Authentication: API key passed in the x-apikey request header.
- Response: a last_analysis_stats object with per-category vendor counts (malicious, suspicious, harmless, undetected) aggregated from roughly 70–90 independent antivirus/security vendor engines.

- Design choice: this implementation only reads existing/cached VirusTotal reports (GET), rather than submitting a fresh scan (POST) and polling for results — this keeps response times acceptable for a live-browsing use case, at the cost of not getting an analysis for URLs VirusTotal has never seen before.

2.8.3 Local heuristics (no external call)

A small set of local checks run without any network call, adding a modest score contribution and surfacing signals the two providers might not catch on a brand-new page:

- Not using HTTPS
- Host is a raw IP address rather than a domain name
- Punycode/IDN hostname (a common homograph/impersonation technique)
- URL contains an "@" character (a classic redirection/obfuscation trick)

2.9 Risk Scoring Logic

The risk engine is intentionally simple and transparent rather than a black box, which matters for a security warning users need to trust:

- Each provider that flags the URL contributes +55 to the score.
- Each triggered heuristic contributes a smaller fixed amount (10–15 points depending on signal strength).
- The total is capped at 100.

Score range	Status
0 – 19	safe
20 – 54	suspicious
55 – 100	malicious

Example response for a clean URL:

```
{
  "url": "http://example.com",
  "score": 10,
  "status": "safe",
  "findings": [
    { "source": "Google Safe Browsing", "flagged": false, "detail": "No known threats found" },
    { "source": "VirusTotal", "flagged": false, "detail": "Clean across 90 vendors" }
  ],
  "heuristics": ["Not using HTTPS"],
  "checkedAt": "2026-09-18T12:22:05.633Z"
}
```

These specific thresholds (55 / 20) are a reasonable starting point, not a tuned or validated model — see Section 2.14 for the limitation this implies.

2.10 Chrome Extension Implementation

File	Responsibility
manifest.json	Manifest V3 declaration: permissions, host permissions, background service worker, and popup registration.
background.js	Service worker. Listens for <code>chrome.webNavigation.onCommitted</code> , calls the backend, updates the toolbar badge, and redirects the tab to <code>warning.html</code> when a URL is flagged.
popup.html / popup.js	On-demand view: shows the current tab's URL, score, status, and findings when the user clicks the extension icon.
warning.html / warning.js	A bundled extension page shown in place of a flagged site. Displays the score and findings, and offers "Go Back" or a one-time "Proceed Anyway" bypass.

Manifest permissions used: `activeTab`, `storage`, `tabs`, and `webNavigation`, plus `host_permissions` scoped to the backend's origin only (not the broad `<all_urls>`). This follows the principle of requesting the minimum access the extension actually needs, though `tabs` itself is still a fairly broad permission — see Section 2.14.

2.11 Manifest V3 Design Considerations

The assignment specifically asked how several Manifest V3 and production-readiness concerns were handled. This section addresses each directly and honestly, including where the current build has a gap rather than a full solution.

Service-worker lifecycle

Manifest V3 background scripts are non-persistent service workers: Chrome can terminate them when idle and wake them again on a registered event (here, `chrome.webNavigation.onCommitted`). Any state that needs to survive a service-worker restart cannot live in an ordinary in-memory JavaScript variable. This is why the one piece of cross-request state the extension needs — the one-time "user chose to proceed anyway" bypass flag — is stored in `chrome.storage.session` rather than a variable, so it survives the service worker being unloaded and reloaded between the warning page's action and the next navigation event.

Content script vs. background/service-worker responsibilities

This implementation deliberately does not use a content script. Instead of injecting a banner or overlay into the existing page's DOM, the background service worker redirects the entire tab to a bundled extension page (`warning.html`) before the risky page is shown. Trade-off, stated plainly: a content-script overlay would let the browser show the warning on top of the original page, but it can be more fragile (a page's own CSS/JS, or a strict page CSP, can interfere with an injected overlay) and it still lets the risky page's network requests and scripts execute underneath it. The full-page redirect used here cannot be interfered with by the page itself, at the cost of not literally being an overlay on the original page — this is a deliberate simplification worth revisiting if pixel-perfect "banner over the page" behavior is required.

Permission scoping

`host_permissions` is limited to the backend's own origin, not `<all_urls>` — the extension does not request blanket access to read every page's content, since it never needs to (it only ever sends the URL string itself, not page content, to the backend). The `tabs` permission is broader than strictly minimal and is used so the

background script can redirect any tab, not just the currently active one; a production version should evaluate whether activeTab alone could satisfy this instead.

API latency and timeouts

Not fully implemented. The current provider adapters (`googleSafeBrowsing.provider.js`, `virusTotal.provider.js`) call axios without an explicit timeout or AbortController, so a slow or hanging upstream API could delay the assessment indefinitely. This is a known gap — see Section 2.14 and 2.15 for the recommended fix (a fixed timeout per provider call, e.g. via axios's timeout option, combined with the existing try/catch fallback).

Failed or unavailable threat-intelligence services

Implemented. `Promise.allSettled` (rather than `Promise.all`) is used in `urlIntelligence.service.js` specifically so that one provider throwing or rejecting does not take down the other's result or the whole request — a failed provider degrades to a Finding of `{ flagged: false, detail: "Source unavailable: ..." }` rather than crashing the endpoint.

Caching and duplicate requests

Not implemented. Every navigation triggers a fresh call to both providers, with no caching layer and no de-duplication of repeated checks of the same URL in a short window. This matters in particular for VirusTotal's free-tier rate limit, and is one of the more important gaps to close before any real usage — see Section 2.15.

False positives and false negatives

Handled only at a structural level, not a validated one: combining two independent sources plus local heuristics reduces (but does not eliminate) the chance that a single source's mistake drives the verdict, and the score is deliberately shown alongside the individual findings so the user (or a developer debugging the system) can see why a verdict was reached rather than trusting an opaque number. No formal precision/recall testing against a labeled set of known-safe and known-malicious URLs has been performed, and the score thresholds (Section 2.9) are a reasonable starting guess, not a tuned model.

Safe handling of untrusted data

Provider responses and their detail strings are rendered into the popup and warning page using `textContent` assignment (e.g. `li.textContent = ...`), not `innerHTML` — this means even if a provider ever returned a string containing HTML/script-like content, it would be displayed as inert text, not parsed and executed. This was a deliberate choice, not an incidental one, since the content shown in these UI surfaces ultimately originates from third-party services.

Basic security considerations for production

- API keys (Google Safe Browsing, VirusTotal) are kept server-side only, in a `.env` file excluded from version control — the extension never sees or holds either key.
- CORS is currently wide open (`app.use(cors())` with no origin restriction) for local development convenience — this must be restricted to the extension's own origin before any real deployment.
- The backend currently runs over plain HTTP on localhost — a production deployment needs HTTPS end-to-end.
- There is no authentication on the `/api/security/check-url` endpoint and no rate limiting — currently anyone who can reach the backend's URL can call it, which is acceptable for local development only.

- Input validation exists (url.validator.js rejects malformed/non-http(s) input) but there is no logging, monitoring, or abuse-detection layer yet.

2.12 Setup & Run Instructions

Backend

```
cd url-risk-checker-backend
npm install
# create a .env file in the project root with:
#   GOOGLE_SAFE_BROWSING_API_KEY=your_key_here
#   VIRUSTOTAL_API_KEY=your_key_here
#   PORT=3000
npm run dev
# server starts at http://localhost:3000
```

Verify it is working:

```
curl -X POST http://localhost:3000/api/security/check-url \
-H "Content-Type: application/json" \
-d "{\"url\":\"http://example.com\"}"
```

Extension

- Open chrome://extensions in Chrome.
- Turn on Developer Mode (top-right toggle).
- Click "Load unpacked" and select the url-risk-checker-extension folder (the folder containing manifest.json — not the backend folder).
- Make sure the backend is running before browsing, since the extension calls http://localhost:3000.
- Navigate to any http(s) page; a flagged page redirects automatically to the warning screen, and clicking the toolbar icon on any page shows the current score and findings on demand.

2.13 AI-Assisted Development Notes

How the work was structured

Development proceeded conversationally and iteratively with Claude, building the project file by file in the order data flows through the system: validator, then each provider adapter, then the aggregation service, then the risk engine, then the controller/route, then server.js, and finally the extension files. Each file was reviewed and tested (via curl, then via the loaded extension) before moving to the next, rather than generating the whole project at once.

What AI assisted with

- Scaffolding every backend file and the extension files (manifest, background service worker, popup, warning page).
- Explaining the exact request/response shape of the Google Safe Browsing and VirusTotal APIs before writing the adapters against them.
- Diagnosing setup issues along the way (npm packages installed globally instead of locally, an empty server.js causing a silent "clean exit", a stale manifest.json referencing a background.js file that didn't exist yet).
- Producing this documentation, including the architecture diagrams.

How AI-generated code was reviewed and validated

- Every backend endpoint was tested manually with curl against real inputs, including a known-clean URL (example.com) and Google's official Safe Browsing test URL, before being considered done.
- The extension was loaded as an unpacked extension in Chrome and exercised against a real page, with the popup's DevTools console checked for errors.
- Each new file's purpose and how it connected to the others (variable naming, module.exports, req/res, the difference between a project-local vs. global npm install) was walked through and confirmed to be understood before proceeding, rather than pasted in blind.

What was rejected or changed along the way

- The initial architecture direction (explored separately in ChatGPT) was an ASP.NET Core backend using Clean Architecture and MediatR. This was deliberately abandoned in favor of a Node.js + Express backend for a simpler, faster build matching the team's stated preference.
- The first working version of the extension only checked the URL on demand, when the user clicked the toolbar icon — this did not satisfy the requirement to warn before the user lands on a risky page. It was replaced with the current background-service-worker approach (chrome.webNavigation.onCommitted) that checks automatically and redirects proactively, with the popup kept as a secondary, on-demand view.
- An early manifest.json still referenced a background.js file left over from an earlier design that had been dropped; this caused a "Could not load background script" error on load and was corrected once identified.

2.14 Known Limitations

- No caching layer — every navigation triggers a fresh call to both providers, with no de-duplication or TTL-based caching.
- No explicit request timeout on the provider HTTP calls — a slow upstream API can delay a result rather than failing fast.
- CORS is currently open to any origin; not scoped to the extension.
- No authentication or rate limiting on the backend endpoint.
- Backend runs on plain HTTP (localhost) rather than HTTPS.
- Risk-score thresholds are a reasonable starting heuristic, not validated against a labeled dataset of known-safe/known-malicious URLs.
- VirusTotal integration only reads existing/cached reports; it does not submit new URLs for scanning, so a URL VirusTotal has never seen returns "not previously scanned" rather than a fresh verdict.
- The warning experience is a full-page redirect rather than an in-page banner/overlay layered on top of the original site.
- No automated test suite (unit tests for the risk engine, integration tests for the endpoint) exists yet — testing so far has been manual (curl, Postman, and the loaded extension).

2.15 What Would Change for Production

- **Add caching:** an in-memory or Redis-backed cache keyed by URL, respecting each provider's own recommended cache duration (Safe Browsing returns a `cacheDuration` field), to cut both latency and API usage.
- **Add timeouts and retries:** a fixed per-provider timeout (e.g. 3–5 seconds) with a clear fallback message, so one slow provider never blocks the whole assessment.
- **Lock down CORS:** restrict allowed origins to the extension's own `chrome-extension://` origin rather than allowing any site to call the API.
- **Add authentication and rate limiting:** an API key or signed token from the extension, plus per-IP/per-key rate limits, so the endpoint cannot be used as an open proxy.
- **Move to HTTPS:** deploy the backend behind TLS (a managed platform or a reverse proxy with a certificate), and update `host_permissions` accordingly.
- **Add structured logging and monitoring:** request volume, provider error rates, and latency, so degraded upstream APIs are visible rather than silent.
- **Add automated tests:** unit tests for `riskEngine.service.js` (pure function, easy to test with fixed inputs) and integration tests for the endpoint using mocked provider responses.
- **Validate the scoring thresholds:** test against a labeled set of known-safe and known-malicious URLs and adjust the point values and thresholds based on measured false-positive/false-negative rates, rather than the current starting-point guess.

2.16 30-Day MVP Roadmap

If given 30 days to move this from proof-of-concept toward an MVP, the recommended priority order is:

Week	Focus
Week 1	Production-readiness hardening: timeouts, caching, CORS lockdown, HTTPS deployment, basic auth on the endpoint.
Week 2	Testing: unit tests for the risk engine, integration tests for the endpoint with mocked providers, and a first pass at validating score thresholds against a small labeled URL set.
Week 3	Extension polish: an in-page content-script warning banner as an alternative/complement to the full-page redirect, plus a simple options page for the user to review past warnings.
Week 4	Add a third signal source (e.g. URLhaus or PhishTank) and a lightweight user-reporting flow, both of which meaningfully improve coverage of newly emerging threats without requiring a proprietary detection engine.

2.17 Time Spent & Attribution

[To be filled in before submission — this section should record actual hours spent, broken down roughly by research vs. backend vs. extension vs. debugging vs. documentation, since only the author has this information.]

What was personally directed vs. AI-generated

All code in this project was AI-generated (primarily via Claude, with ChatGPT used for early architecture exploration) in direct response to specific, incremental instructions and decisions made by the author at each step — including the choice of tech stack, the decision to add proactive background-based checking instead of

only on-click checking, and the verification of every endpoint and extension behavior before moving forward.
[Add any specifics here: which exact prompts or sessions took the most iteration, and any code written or modified by hand rather than generated.]

Appendix A — Source Repository

[<https://github.com/Acmasule777/Manifest-V3-Extension-URL-Risk-Checker.git>]

Repository structure:

[Manifest-V3-Extension-URL-Risk-Checker/](#)

```
url-risk-checker-backend/  
  src/  
    routes/security.routes.js  
    controllers/security.controller.js  
    validators/url.validator.js  
    services/urlIntelligence.service.js  
    services/riskEngine.service.js  
    providers/googleSafeBrowsing.provider.js  
    providers/virusTotal.provider.js  
    config/env.js  
  server.js  
  package.json  
  .env          (not committed)  
  .gitignore  
  
url-risk-checker-extension/  
  manifest.json  
  background.js  
  popup.html  
  popup.js  
  warning.html  
  warning.js
```